
B28: SQL SuperHero Program : Session 28

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

1 April 2025 at 22:23

FUNCTION And PROCEDURE

Function:

PL/SQL function is a reusable program unit stored as a schema object in the Oracle Database.

Use the CREATE FUNCTION statement to create a standalone stored function or a call specification.

A stored function (also called a user function or user-defined function) is a set of PL/SQL statements you can call by name. Stored functions are very similar to procedures, except that a function returns a value to the environment in which it is called. User functions can be used as part of a SQL expression. (Means we can call in a select)

Procedure:

A PL/SQL procedure is a reusable unit that encapsulates specific business logic of the application. Technically speaking, a PL/SQL procedure is a named block stored as a schema object in the Oracle Database. A procedure is a group of PL/SQL statements that you can call by name.

diff between procedure and function:

1. Function is to compute something and procedure is to execute something
2. Function must return a value and Procedure may or may not
3. While creating function, we must have a Return clause else it gives compile error and there is nothing like that in procedure
4. We can use function in a select statement but we can not use procedure in select statement

Write a Function which will give you salary of the employee:

```
create or replace function get_sal(p_id number) Return number AS
L_salary Number;
Begin
  select salary into L_salary from hr.employees where employee_id = p_id;
  Return L_salary;
End get_sal;
```

Syntax

```

1 create or replace function get_sal(p_id number) Return number AS
2   l_salary Number;
3 Begin
4   select salary into l_salary from hr.employees where employee_id = p_id;
5   Return l_salary;
6 End get_sal;

```

Annotations:

- name of fu (Any name)**: Points to `get_sal`
- where clause of sql = input parameter**: Points to `p_id`
- Data type of input**: Points to `number`
- write your SQL first**: Points to the SQL query in line 4
- mandatory (else compile error)**: Points to the `Return` statement in line 5
- mandatory (else run time error)**: Points to the `Return` statement in line 5
- convert SQL to PL/SQL**: Points to the entire function definition

Function created.

See how we write a function :

1. write your sql
2. convert your sql to pl/sql
3. write the definition line
create or replce function --> syntax
get_sal --> any name of function
() --> bracket if we have any where clause in query and we want to make it dynamic.
p_id --> it is actually employee_id of query
number--> data type of input
and return is mandatory else give compile error
next number is datatype of output. my function return salary and salary is number, so it is number

where clause

```

create or replace function get_email(p_employ_id number) return varchar2 as
l_email hr.employees.email%type;
Begin
  select email into l_email from hr.employees where employee_id = p_employ_id;
  Return l_email;
End;

```

Function created.

MANDATORY

Name can be anything

```

create or replace function reeta(p_employ_id number) return varchar2 as
l_email hr.employees.email%type;
Begin
  select email into l_email from hr.employees where employee_id = p_employ_id;
  Return l_email;
End;

```

Function created.

for Return output

function with 1 parameter:

```
create or replace function get_region_name(P_region_id IN NUMBER) Return Varchar2 AS
L_Region_Name Varchar2(20);
Begin
  Select Region_Name Into L_Region_Name from regions where region_id = P_region_id;

  RReturn L_Region_Name;

End get_region_name;
```

use of the function:

```
select get_region_name(1) from dual;
```

The screenshot shows the SQL Developer interface with a function definition and its execution. Handwritten blue annotations explain the components:

- ① **datatype of output**: points to the `return varchar2` in the function definition.
- ② **datatype input**: points to the `NUMBER` in the parameter `P_region_id`.
- ③: points to the `return` keyword.
- ④ **where clause**: points to the `where employee_id = vishal` in the `select` statement.
- ⑤ **must return**: points to the `return l_name;` line.

The execution results show the function was created successfully and the query `select get_name(100) from dual` returned the value `Steven`.

Check the above example. Start with

1. **Varchar2** --> this is datatype of my output. we wrote this function to give me first_name. means , we are expecting first name as output. first name is a varchar2 type, so my function return varchar2
2. **number** --> this is the datatype of my input. input means where clause. what my function expects. it is expecting the employee id, so what is the datatype of that, we have to write here.
3. **return** --> we must have this return, else function gives compile time error.
4. **vishal** --> this is your where clause. name can be anything, but this dynamic, whatever we pass it pass to query. just like in example we pass 100, value of vishal is 100 and it passes to the sql function.
5. **return** --> this is for return. function must return a value. so this is what the function is returning.

create or replace function nagaraju (sanu number) return varchar2 as
 l_email hr.employees.email%type;
 Begin
 select email into l_email from hr.employees where employee_id = sanu;
 Return l_email;
 End;

Function created.

create or replace function vishal (p_employee_id number) return date as
 l_salary hr.employees.hire_date%type;
 Begin
 select hire_date into l_salary from hr.employees where employee_id = p_employee_id;
 return l_salary;
 End;

Function created.

select vishal(101) from dual

VISHAL(101)

21-SEP-05

create or replace function reeta(p_employ_id number) as --return varchar2 as
 l_email hr.employees.email%type;
 Begin
 select email into l_email from hr.employees where employee_id = p_employ_id;
 Return l_email;
 End;

create or replace function reeta(p_employ_id number) as --return varchar2 as
l_email hr.employees.email%type;
Begin
select email into l_email from hr.employees where employee_id = p_employ_id;
Return l_email;
End;

Errors: FUNCTION REETA
Line/Col: 1/36 PLS-00103: Encountered the symbol "AS" when expecting one of the following:

return

create or replace function reeta(p_employ_id number) return varchar2 as
l_email hr.employees.email%type;
Begin
select email into l_email from hr.employees where employee_id = p_employ_id;
-- Return l_email;
End;

Function created.

create or replace function reeta(p_employ_id number) return varchar2 as
l_email hr.employees.email%type;
Begin
select email into l_email from hr.employees where employee_id = p_employ_id;
-- Return l_email;
End;

BUT when we call this function, it gives an error.

```
select reeta(102) from dual
```

```
ORA-06503: PL/SQL: Function returned without value
```

=====

See the below code. We want to write a function that if we pass employee id, function gives me department_name.

1. first write your sql
2. convert in pl/sql
3. understand here: we want to return department_name, its varchar2, so function return varchar2

```
create or replace function get_department(p_id number) return varchar2 as
```

```
l_dept hr.departments.DEPARTMENT_NAME%Type;
```

```
Begin
```

```
select DEPARTMENT_NAME into l_dept
```

```
from hr.departments d, hr.employees e
```

```
where d.department_id = e.department_id
```

```
and e.employee_id = p_id;
```

```
return l_dept;
```

```
End;
```

```
select get_department(110) from dual;
```